

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_28

28. Criticisms of Design Patterns

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

Design patterns help you benefit from other people's experiences. This is often termed *experience reuse*. You learn how they solved challenges, how they tried to adopt new behaviors in their systems, and so on. A pattern may not perfectly fit into your work, but if you know the best practices as well as the problems of a pattern at the beginning, you are more likely to make a better application. This is why I now tell you about the criticisms of design patterns. Knowing about them can offer you some real value. If you think critically about these patterns before you design your software, you can predict the return on investment (ROI) to some degree. So, let's go through the following points, which are often raised by some developers:

- The concept of patterns came through Christopher Alexander. He was an architect, not a computer programmer. He considered a domain that did not change a lot over the years (compared to the software industry). On the contrary, the software industry is always changing so changes to software development are much faster compared to other domains. This is why critics often say that you cannot start from the domain of buildings and towns that Christopher Alexander considered.
- The way you used to write a program in the early days is not comparable to today. The way of writing codes has changed a lot. In today's world, computer languages support many built-in advanced features as well. So, the facilities that you have nowadays are much more compared to the old days of programming. So, when you extract patterns based on old practices, you show additional respect to them.
- Many of these patterns are similar, and there are always pros and cons associated with each pattern (I discuss them in the Q&A sessions

at the end of each chapter). A pitfall in one case can be a real virtue in a different case.

- The pattern that supplies satisfactory results today can be a big burden in the near future due to the continuous changes in the software industry.
- It is very unlikely that the infinite number of requirements can be well designed with these finite number of design patterns.
- Designing software is an art. And there is no definition or criteria for the best art.
- Design patterns give you the idea but not the implementation (like libraries and frameworks). You know that each human mind is unique. So, each engineer may have preferences for implementing a similar concept, and this can create chaos on a team if mindsets vary widely.
- Consider a simple example. Patterns encourage people to code to a supertype (abstract class/interface). But for a simple application where you know that there is no upcoming change in the near future, or an application created for a demo purpose only, this idea may not make much sense to you.
- Similarly, in smaller applications, you may find that enforcing the rules of design patterns increases your code size and maintenance costs.
- Erasing the old and adopting the new is not always easy. For example, when you first learned about inheritance, you were excited. You probably wanted to use it in many ways and saw only the benefits from the concept. But later when you started experimenting with design patterns, you learned that in many cases compositions are preferred over inheritance. This shifting of programming mindsets is not easy.
- Design patterns are based on key principles, and one of them is to *identify the code that may vary and then separate it from the rest of the code*. This sounds very good from a theoretical perspective. But in real-world implementations, who guarantees that your judgment is perfect? The software industry always changes, and it needs to adapt to new requirements/demands continuously.
- Many patterns are already integrated with modern-day languages. Instead of implementing the pattern from scratch, you can use the built-in support in the language constructs.

- Inappropriate use of patterns can lead to anti-patterns. For example, an inappropriate use of the Mediator Pattern can lead to the God Class anti-pattern. I provide a brief overview of anti-patterns in Chapter [29](#).
- Many people believe that the concepts of design patterns simply indicate that a programming language may need additional features. So, patterns will have less significance with the increasing capability of modern-day programming languages. Wikipedia says that computer scientist Peter Norvig believes that 16 out of the 23 GoF design patterns are simplified or eliminated via direct language support in Lisp or Dylan. You can see similar thoughts at https://en.wikipedia.org/wiki/Software_design_pattern.
- The patterns I discuss in this book are solely based on object-oriented programming. The efficiency and applicability of these patterns are questionable in other domains.
- These patterns are not interchangeable.
- In the end, design patterns help you to get benefit from the experiences of others. You are getting their thoughts, you come to know how they encountered the challenges, how they implemented new behaviors in their systems, and more. But if you dive deep down to the basic thought, you will find that they start with the assumption that a beginner or relatively less experienced person cannot solve a problem better than their seniors. On some specific occasions, though, a relatively less experienced person can have a better vision than their seniors and become more effective in the future.

Q&A Session

28.1 Is there any catalog for these patterns?

I started with GoF's 23 design patterns and then discussed more patterns in this book. The GoF catalog is considered the most fundamental pattern catalog.

But definitely, there are many other catalogs that may focus on particular domains. The Portland Patterns Repository and The Hillside Group's website are well known in this context. You can get valuable insights and thoughts from these resources. As a starting point, you can browse through the following links:

<https://wiki.c2.com/?PortlandPatternRepository>.

<https://hillside.net/patterns/patterns-catalog>

From the Hillside group website, you can learn about various conferences and workshops also.

NoteAt the time of writing, the links in this book worked, but these links and policies to access these links may change in the future.

28.2 Why are you silent about other patterns?

These are my personal beliefs:

- Computer science will keep growing, and you will keep getting new patterns.
- If you are not familiar with the fundamental patterns, you cannot evaluate the true needs of the remaining or upcoming patterns. For example, if you know MVC well, you can see how it is different from Model-View-Presenter (MVP) or Model-View-ViewModel (MVVM) and understand why they are needed.
- The book is already fat. A detailed discussion of each pattern would need many more pages, which would make the book too big to digest.

So, in this book, I focus on fundamental patterns that are still relevant in today's programming world.

28.3 I often see the term “force” with the description of design patterns. What does it mean?

It is the criteria on which developers justify their developments. Broadly, your target and current constraints are two important parts of your force. Therefore, when you develop your application, you can justify your development with these parts.

28.4 In various forums, I have seen that people are fighting about the pattern definition and say something like this: “A pattern is a proven solution to a problem in a context.” What does this mean?

This is a simple and easy-to-remember definition of what a pattern is. But simply breaking it down into three parts (problem, context, and solution) is not enough.

As an example, suppose you are headed to an airport and you are in a hurry. Suddenly, you discover that you have left your boarding pass at home. Let's analyze the situation:

Problem: You need to reach the airport on time.

Context: You left the boarding pass at home.

The solution that may come into your mind: Turn back and rush toward home at a high speed to get the boarding pass.

This solution may work once, but can you apply the same procedure repeatedly? You know the answer. This is not an intelligent solution because it depends on how much time you currently have to collect the pass from home and go back to the airport. It also depends on the current traffic on the road and many other factors. So, even if you're successful once, you may want prepare a better solution for a similar situation in the future.

So, try to understand the meaning, intent, and context to understand a pattern clearly.

28.5 Sometimes I am confused by similar UML diagrams for two different patterns. I am further confused with the classification of the patterns in many cases. How can I overcome this?

This is perfectly natural. The more you read and analyze the implementations and the more you try to understand the intent behind these designs, the distinctions among them will be clearer to you.

Lastly and most importantly, following a pattern is not your ultimate goal. Your primary goal is to produce an efficient solution that is simple, efficient, and serves your current need.

28.6 When should I consider writing a new pattern?

Writing a new pattern is not easy. You need to study a lot and evaluate the available patterns before you make such an attempt. But if you do not find an existing pattern to serve your domain-specific need, you may need to write your own pattern. It will be very good if your solution passes the “Rule of Three” test, which simply says that to get the tag “pattern,” a solution needs to be applied in a real-world solution at least three times successfully. Once you have done this, you can let others know about it, participate in discussion forums, and get feedback from others. This activity can help both you and the development community.

28.7 Can you name a few patterns outside the GoF’s catalog?

Here are two that you commonly see in other patterns:

- **Utility pattern:** A utility class with static methods only. You avoid the instantiation of this class using a private constructor.
- **Lazy Instantiation pattern:** It delays the object instantiation process until it is truly required. It is useful to deal with resource-hungry/resource-heavy objects.

Previous chapter

< [Part IV. The Final Talks on Design Patterns](#)

Next chapter

[29. Anti-Patterns](#) >